

Documento de Design

Blog Pessoal

Backend em Java (Spring Boot), front em Next.js, MongoDB e Docker.

Autor: William

Versão 1.0

1. Visão geral

Aplicação web de blog pessoal com área pública (leitura) e área administrativa (autor). O conteúdo se organiza em três eixos: Sobre mim, Projetos e Estudos (posts). Há ainda uma seção de Vídeos, cujos arquivos ficam hospedados no YouTube — o sistema guarda apenas o ID/URL do vídeo e incorpora o player no navegador.

Objetivos

- Publicar e gerenciar posts, projetos e vídeos sem precisar mexer em código.
- SEO decente para os posts (URLs por slug, renderização amigável a crawlers).
- Ser simples de subir e rodar local e em produção com um único docker compose up.

Não-objetivos (v1)

- Multiusuário / múltiplos autores (apenas um administrador).
- Comentários, newsletter, likes (evolução futura).
- Editor WYSIWYG complexo — o conteúdo é escrito em Markdown.

2. Stack e decisões de arquitetura

Camada	Escolha	Por quê
Backend	Java 17 + Spring Boot 3	Requisito do projeto; ecossistema maduro para API REST.
Front	Next.js (React) via REST	SSG/SSR SEO para blog.
Banco	MongoDB 7	Banco de documentos; flexível para conteúdo de formato variável.
Auth	JWT (login do admin)	Simples e stateless; suficiente para um único usuário.
Orquestração	Docker + Compose	Requisito; três serviços (web, api, db).
Migrações	Mongock (opcional)	Evoluções de dados versionadas quando necessário.

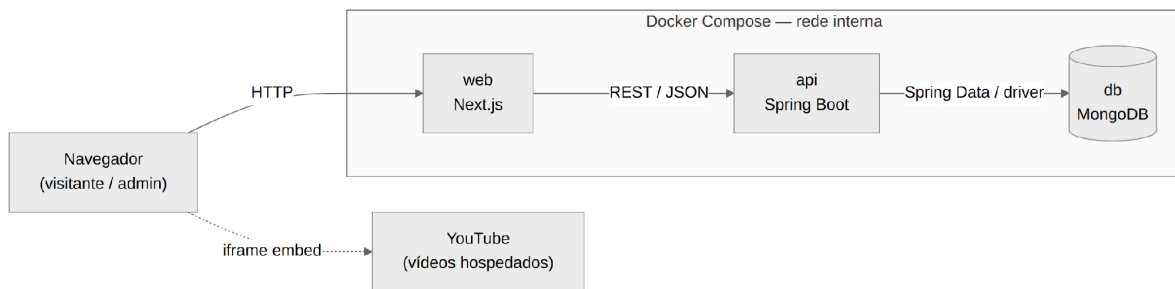
Decisão-chave: front desacoplado vs. monolito

A proposta principal é desacoplada (web Next.js + api Spring Boot separados): melhor SEO/performance no front, separação de responsabilidades e uso dos seus pontos fortes.

Alternativa mais simples: monolito Spring Boot + Thymeleaf, servindo HTML direto do Java. Elimina o container web e o CORS, ao custo de um front menos flexível. Nesse caso o modelo de domínio e o de dados não mudam — muda só a camada de apresentação.

3. Arquitetura de containers

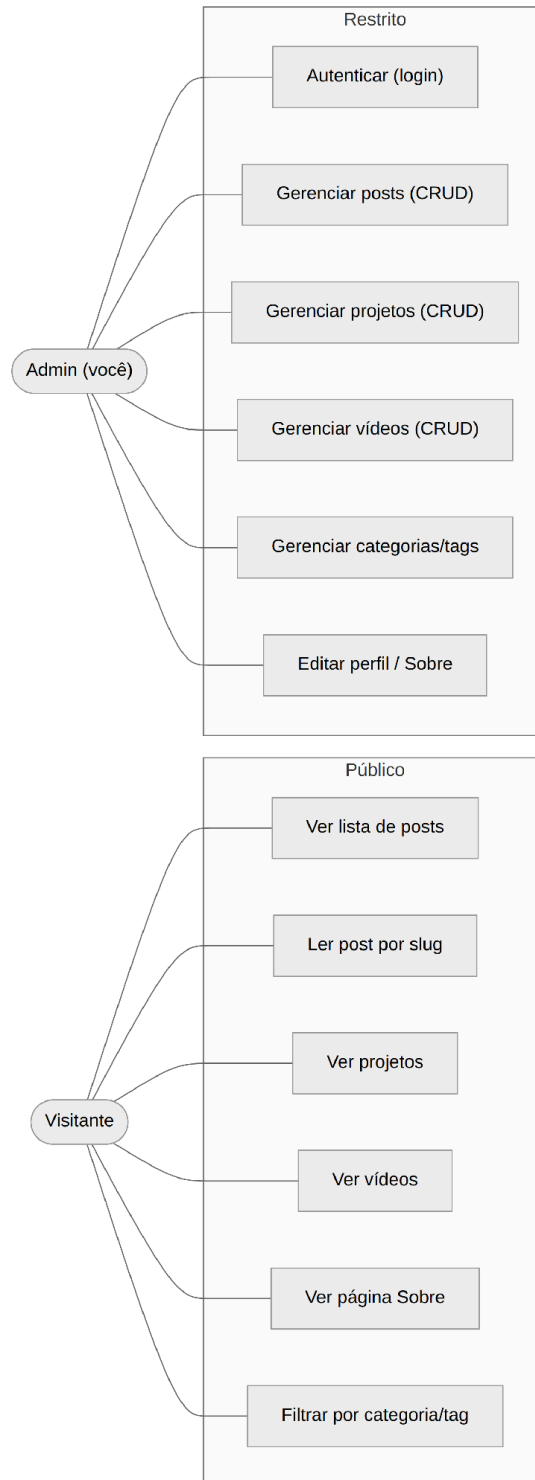
Três serviços orquestrados por Docker Compose. O front consome a API por REST na rede interna; o banco só é acessível internamente.



- web renderiza as páginas e chama api pela rede interna do Compose.
- api é a única a falar com o banco.
- db não expõe porta para fora em produção — só na rede interna.
- Os vídeos são incorporados client-side; o banco guarda apenas o youtube_id.

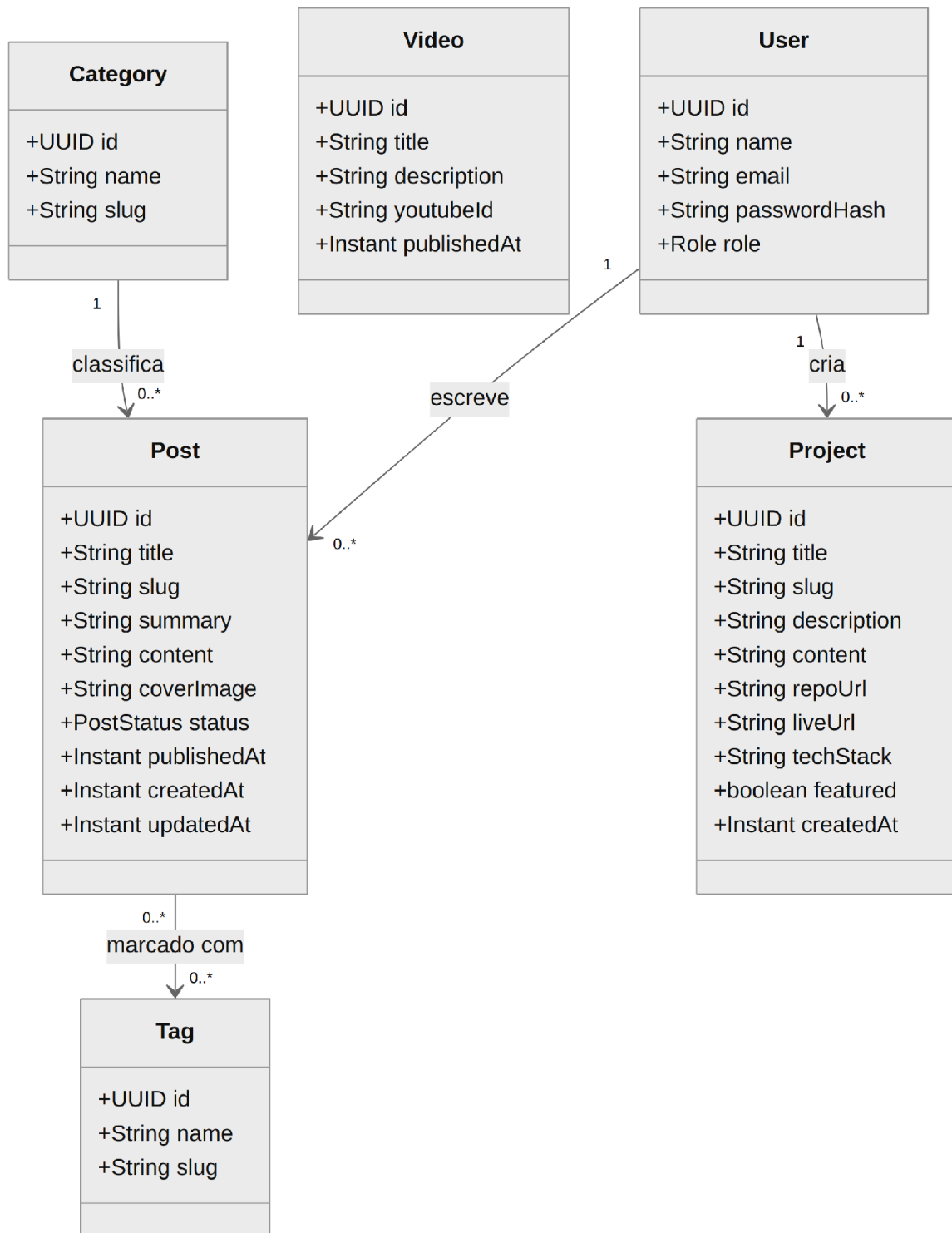
4. Requisitos funcionais

Casos de uso separados por ator. Tudo em Público é aberto (GET); tudo em Restrito exige JWT válido de admin.



5. Modelo de domínio

Entidades principais e seus relacionamentos.



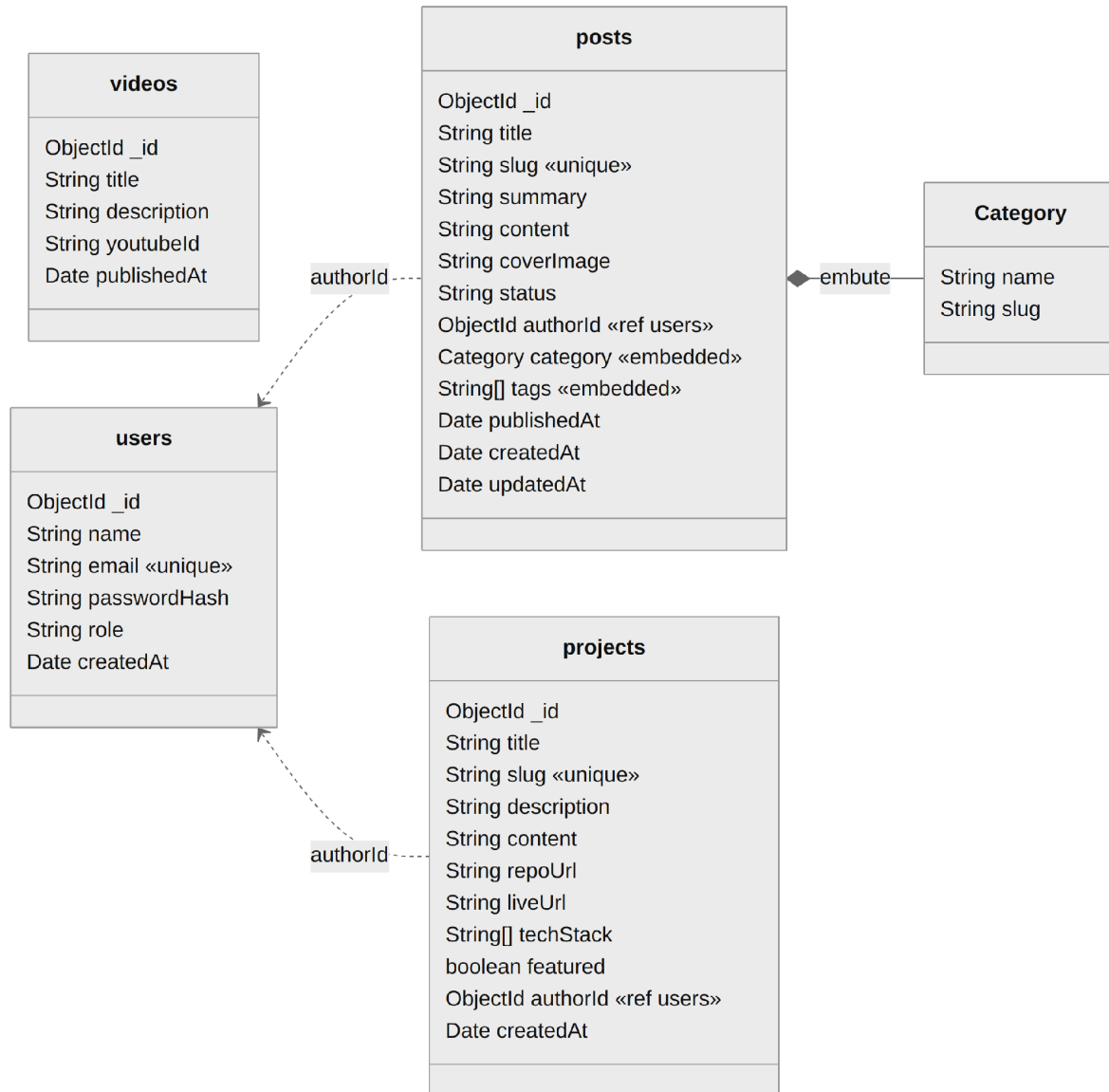
content de Post/Project é Markdown, renderizado no front.

PostStatus = DRAFT | PUBLISHED. Rascunhos não aparecem na área pública.

Video é entidade independente. Para vincular a um post/projeto no futuro, basta um FK opcional.

6. Modelo de dados

Modelo orientado a documentos. Cada post guarda a categoria e as etiquetas embutidas no próprio documento (a categoria como subdocumento, as etiquetas como um vetor de strings), o que dispensa tabelas associativas. O autor é mantido como referência (authorId). Os slugs recebem índice único, pois compõem as URLs.



Índices sugeridos: único em `posts.slug` e `users.email`; composto em `posts(status, publishedAt)` para a listagem pública.

7. API REST (esboço)

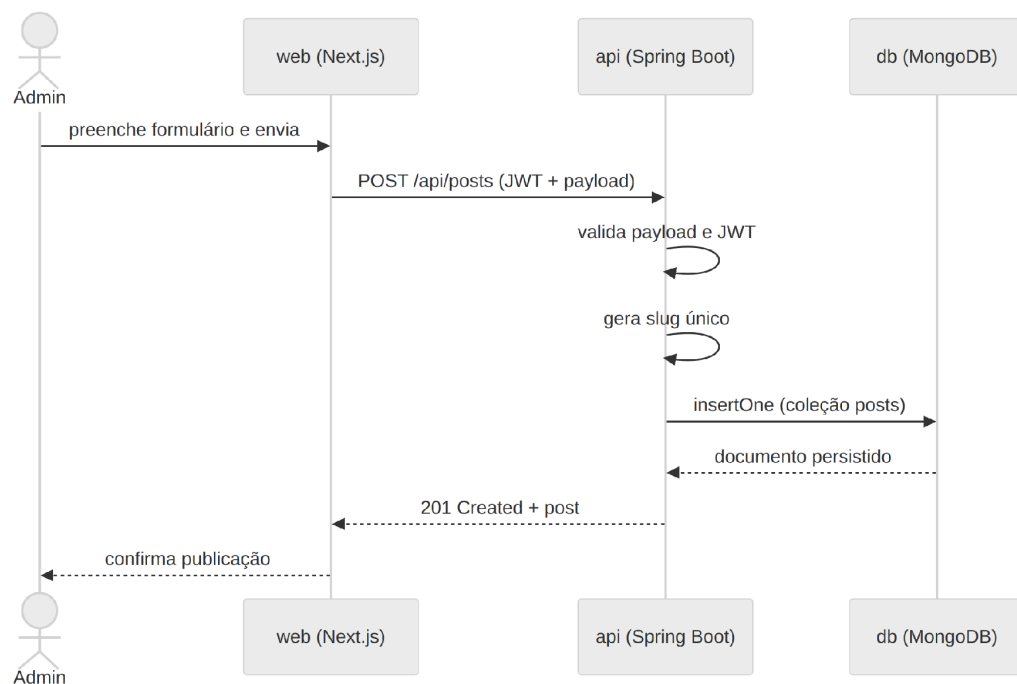
Base /api. Respostas em JSON. Rotas de escrita exigem Authorization: Bearer <jwt>.

Método	Rota	Acesso	Descrição
POST	/auth/login	público	Retorna JWT.
GET	/posts	público	Lista posts publicados (paginado, filtros).
GET	/posts/{slug}	público	Detalhe de um post.
POST/PUT/DELETE	/posts...	admin	CRUD de posts.
GET	/projects	público	Lista projetos.
GET	/projects/{slug}	público	Detalhe de projeto.
POST/PUT/DELETE	/projects...	admin	CRUD de projetos.
GET	/videos	público	Lista vídeos.
POST/PUT/DELETE	/videos...	admin	CRUD de vídeos.
GET	/categories, /tags	público	Listagens auxiliares.
GET	/profile	público	Dados da página Sobre.

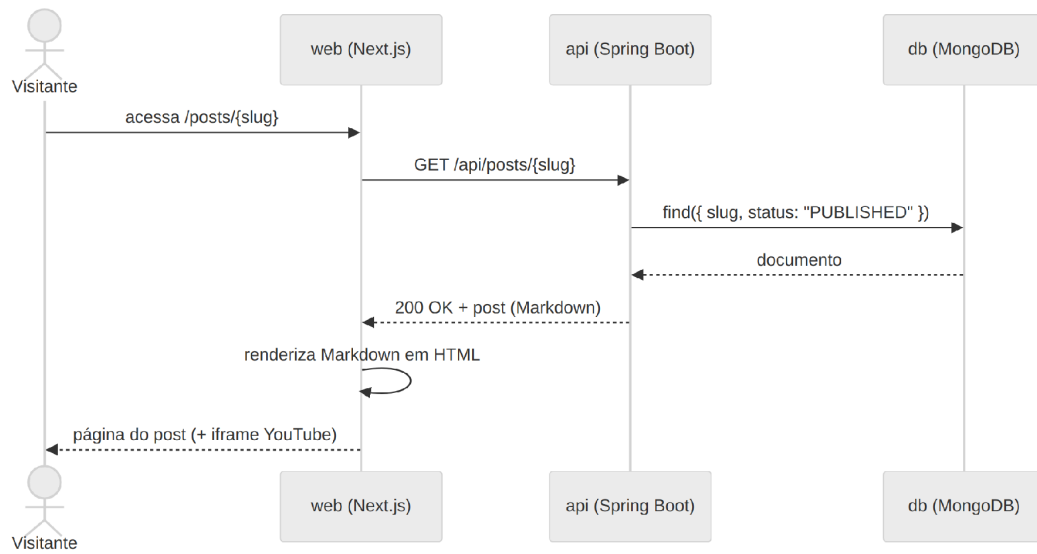
Convenções: paginação com ?page=&size=; validação com jakarta.validation; erros no formato padrão do Spring.

8. Fluxos principais

8.1 Publicar um post



8.2 Visitante lê um post



9. Docker Compose

Esboço dos serviços (ajustar variáveis de ambiente e portas conforme necessário).

```
services:
  db:
    image: mongo:7
    environment:
      MONGO_INITDB_ROOT_USERNAME: blog
      MONGO_INITDB_ROOT_PASSWORD: ${DB_PASSWORD}
      MONGO_INITDB_DATABASE: blog
    volumes:
      - db_data:/data/db
    # sem "ports" em producao - acesso so pela rede interna

  api:
    build: ./api
    environment:
      SPRING_DATA_MONGODB_URI:
mongodbs://blog:${DB_PASSWORD}@db:27017/blog?authSource=admin
      JWT_SECRET: ${JWT_SECRET}
    depends_on: [db]
    ports: ["8080:8080"]

  web:
    build: ./web
    environment:
      NEXT_PUBLIC_API_URL: http://localhost:8080/api
    depends_on: [api]
    ports: ["3000:3000"]

volumes:
  db_data:
```

Em produção, um reverse proxy (Nginx/Traefik) na frente de web e api é o próximo passo — não é necessário para a v1.

10. Estrutura de pastas sugerida

```
blog/
├── docker-compose.yml
├── .env # DB_PASSWORD, JWT_SECRET (fora do git)
├── api/ # Spring Boot
│   ├── Dockerfile
│   └── src/main/java/.../
│       ├── domain/ # documentos (@Document)
│       ├── repository/ # Spring Data
│       ├── service/ # regras de negocio
│       ├── web/ # controllers REST + DTOs
│       ├── security/ # JWT, filtros, config
│       └── config/
├── src/main/resources/ # application.yml, config
├── web/ # Next.js
│   ├── Dockerfile
│   └── src/
│       ├── app/ # rotas (posts, projetos, videos, sobre, admin)
│       ├── components/
│       └── lib/api.ts # cliente da API
```

12. Evoluções futuras

- Comentários (tabela própria + moderação) ou serviço externo.
- Busca nos posts com índices de texto do MongoDB.
- Upload de imagens de capa para um bucket (S3).
- Newsletter / feed RSS.
- Reverse proxy + HTTPS em produção.